

Лабораторная работа № 11

Почтовая инфраструктура Linux и контейнерная платформа Mailcow

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - подготовленный сервер Mailcow;
 - клиентская VM client01;
 - внутренний DNS-сервер;
 - Docker Engine;
 - Docker Compose;
 - Webmail;
 - OpenSSL;
 - swaks;
 - curl;
 - dig.
-

1. Цель работы

Научиться проверять и диагностировать внутреннюю почтовую инфраструктуру на базе Mailcow.

В ходе работы необходимо:

- проверить FQDN, IP-адрес и системное время;
- проверить внутренние DNS-записи A и MX;
- определить состав контейнеров Mailcow;
- зафиксировать версии Mailcow, Docker и Compose;
- создать почтовый домен;
- создать два пользовательских Mailbox;
- проверить вход в Webmail;
- отправить и получить внутреннее письмо;
- проверить SMTP, Submission, IMAPS и HTTPS;
- проверить SMTP Authentication;
- проанализировать очередь Postfix;

- проверить запрет Open Relay;
 - исследовать заголовки письма;
 - смоделировать отказ Dovecot;
 - восстановить сервис и повторить Mail Flow Test.
-

2. Ограничение лабораторной работы

В лаборатории используется внутренний зарезервированный домен:

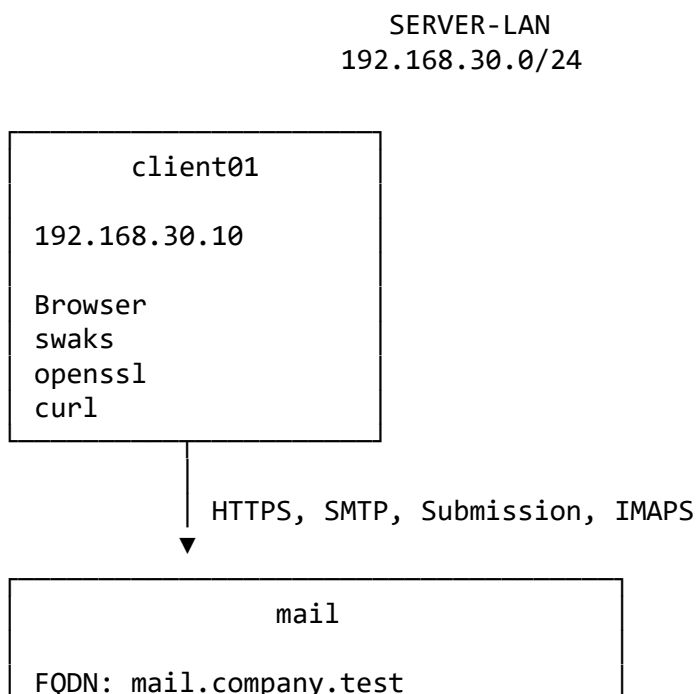
`company.test`

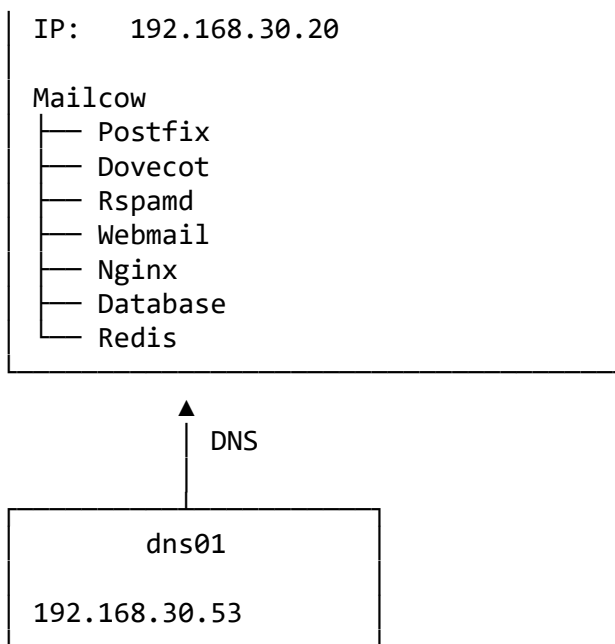
Он подходит для изолированной учебной сети, но не используется для реальной почты в Интернете.

В работе не проверяются:

- доставка в Gmail, Outlook и другие публичные системы;
 - публичная репутация IP-адреса;
 - реальные PTR-записи провайдера;
 - публичные Blocklists;
 - ограничения облачного провайдера на TCP 25;
 - реальная внешняя доставляемость.
-

3. Исходная схема





4. Таблица адресации

Устройство	IP-адрес	Назначение
client01	192.168.30.10	Почтовый клиент и диагностика
mail.company.test	192.168.30.20	Mailcow
dns01	192.168.30.53	Внутренний DNS-сервер

Почтовые объекты

Объект	Значение
Hostname	mail
FQDN	mail.company.test
Mail Domain	company.test
Mailbox 1	user1@company.test
Mailbox 2	user2@company.test

5. Требования к подготовленному стенду

Преподаватель заранее:

- разворачивает Mailcow штатной процедурой для выбранной версии;
- сообщает каталог установки Mailcow;
- предоставляет административную учётную запись;

- настраивает доступ к внутреннему DNS;
- создаёт или импортирует учебный TLS-сертификат;
- обеспечивает разрешение имени mail.company.test;
- проверяет отсутствие конфликтующих сервисов;
- создаёт Snapshot VM перед лабораторной работой.

Пример каталога установки:

```
/opt/mailcow-dockerized
```

Фактический путь сообщает преподаватель.

Универсальные команды установки Mailcow в работе не приводятся, поскольку процедура, состав контейнеров, Backup и Update зависят от используемой версии платформы.

6. Основные почтовые порты

Порт	Протокол	Назначение
25/TCP	SMTP	Обмен почтой между MTA
587/TCP	Submission	Отправка пользователем с STARTTLS и Authentication
465/TCP	SMTPS	Submission через Implicit TLS
143/TCP	IMAP	Доступ к Mailbox, обычно с STARTTLS
993/TCP	IMAPS	Доступ к Mailbox через Implicit TLS
443/TCP	HTTPS	Webmail и административный интерфейс

В базовой работе пользователь отправляет почту через Submission, а читает через Webmail или IMAPS.

7. Базовые задания

Задание 1. Проверить имя сервера, сеть и время

На сервере Mailcow выполнить:

```
hostnamectl  
hostname  
hostname -f
```

Ожидаемый FQDN:

```
mail.company.test
```

Проверить адреса:

```
ip -br link  
ip -br addr  
ip route
```

Проверить адрес сервера:

```
192.168.30.20/24
```

Проверить время:

```
date  
timedatectl
```

Проверить синхронизацию:

```
timedatectl show \  
  --property=NTPSynchronized \  
  --property=TimeUsec \  
  --property=Timezone
```

Ожидается:

```
NTPSynchronized=yes
```

На client01 проверить доступность:

```
ping -c 4 192.168.30.20
```

Проверить решение маршрутизации:

```
ip route get 192.168.30.20
```

Важно

Если Mailcow уже развёрнут, а `hostname -f` возвращает другое имя, нельзя бездумно переименовывать работающий сервер. Несогласованность нужно зафиксировать и передать преподавателю.

FQDN связан с:

- Docker-конфигурацией;
- TLS-сертификатом;
- Webmail;
- DNS;

- SMTP Banner;
 - настройками почтовых клиентов.
-

Задание 2. Проверить отсутствие конфликтующих сервисов

На сервере Mailcow посмотреть прослушиваемые порты:

```
sudo ss -lntup
```

Отдельно проверить основные порты:

```
sudo ss -lntup |  
grep -E ':(25|80|443|465|587|993)\b'
```

Проверить работающие системные службы:

```
systemctl \  
--type=service \  
--state=running \  
--no-pager
```

Проверить локальные службы, которые могут конфликтовать с Mailcow:

```
systemctl is-active postfix 2>/dev/null || true  
systemctl is-active dovecot 2>/dev/null || true  
systemctl is-active nginx 2>/dev/null || true  
systemctl is-active apache2 2>/dev/null || true
```

Проверить контейнеры:

```
sudo docker ps
```

Ожидаемый результат

Почтовые порты принадлежат контейнерной платформе Mailcow, а не отдельному Postfix, Dovecot или Nginx на хосте.

Задание 3. Зафиксировать версии и состав Mailcow

Перейти в каталог Mailcow:

```
cd /opt/mailcow-dockerized
```

Если используется другой путь, заменить его на путь, предоставленный преподавателем.

Проверить версии:

```
sudo docker version  
sudo docker compose version
```

Зафиксировать текущий Git Commit платформы:

```
git rev-parse --short HEAD
```

Посмотреть текущую ветку или Tag:

```
git branch --show-current  
git describe --tags --always
```

Проверить FQDN из конфигурации:

```
grep '^MAILCOW_HOSTNAME=' mailcow.conf
```

Ожидается:

```
MAILCOW_HOSTNAME=mail.company.test
```

Посмотреть сервисы проекта:

```
sudo docker compose config --services
```

Проверить состояние контейнеров:

```
sudo docker compose ps
```

В составе платформы должны определяться роли:

- Postfix;
- Dovecot;
- Rspamd;
- Nginx;
- Webmail;
- Database;
- Redis.

Точные имена сервисов зависят от версии Mailcow.

Создать переменные для дальнейшей диагностики:

```
POSTFIX_SERVICE=$(  
  sudo docker compose config --services |  
  grep -m1 postfix  
)
```

```
DOVECOT_SERVICE=$(  
  sudo docker compose config --services |  
  grep -m1 dovecot  
)
```

```
RSPAMD_SERVICE=$(  
  sudo docker compose config --services |  
  grep -m1 rspamd  
)
```

Проверить:

```
printf 'Postfix: %s\n' "$POSTFIX_SERVICE"  
printf 'Dovecot: %s\n' "$DOVECOT_SERVICE"  
printf 'Rspamd: %s\n' "$RSPAMD_SERVICE"
```

Задание 4. Проверить внутренний DNS

На client01 установить диагностические программы:

```
sudo apt update  
sudo apt install -y \  
    dnsutils \  
    curl \  
    openssl \  
    swaks
```

Проверить A-запись:

```
dig @192.168.30.53 \  
    mail.company.test \  
    A
```

Краткий вывод:

```
dig @192.168.30.53 \  
    +short \  
    mail.company.test \  
    A
```

Ожидается:

192.168.30.20

Проверить MX:

```
dig @192.168.30.53 \  
    company.test \  
    MX
```

Краткий вывод:

```
dig @192.168.30.53 \  
    +short \  
    company.test \  
    MX
```

Ожидается запись вида:

10 mail.company.test.

Проверить системное разрешение имени:

```
getent hosts mail.company.test
```


Проверить почтовый сервер, указанный в MX:

```
MX_HOST=$(
  dig @192.168.30.53 \
    +short \
    company.test \
    MX |
  sort -n |
  awk 'NR==1 {print $2}'
)

printf 'MX host: %s\n' "$MX_HOST"

dig @192.168.30.53 \
  +short \
  "$MX_HOST" \
  A
```

Правильная логика DNS

```
company.test
├── MX 10 mail.company.test
│   └── A 192.168.30.20
```

MX указывает на имя почтового сервера, а не непосредственно на IP-адрес.

Задание 5. Создать домен и два Mailbox

Открыть административный интерфейс:

<https://mail.company.test>

Если используется учебный сертификат от внутреннего СА, доверие к нему должно быть настроено преподавателем.

В административном интерфейсе открыть раздел управления почтовыми доменами.

Названия разделов могут немного отличаться в разных версиях Mailcow.

Создать Mail Domain:

Поле	Значение
Domain	company.test
Description	Internal laboratory mail domain
Active	Да
Mailbox limit	Не менее 2
Default quota	По заданию преподавателя

Создать Mailbox 1:

Поле	Значение
Local part	user1
Domain	company.test
Full address	user1@company.test
Display name	User One
Active	Да

Создать Mailbox 2:

Поле	Значение
Local part	user2
Domain	company.test
Full address	user2@company.test
Display name	User Two
Active	Да

Пароли:

- должны отличаться;
- не включаются в отчёт;
- не сохраняются в Git;
- не записываются в открытый текстовый файл;
- не демонстрируются на скриншотах.

Проверка

В административном интерфейсе должны отображаться:

company.test
user1@company.test
user2@company.test

Задание 6. Проверить Webmail и внутренний Mail Flow

Открыть Webmail по ссылке из административного интерфейса.

В подготовленной версии Mailcow это может быть адрес:

<https://mail.company.test/SOGo/>

Войти как:

user1@company.test

Создать письмо:

Поле	Значение
To	user2@company.test
Subject	LAB11 internal mail flow
Body	Test message from user1 to user2

Зафиксировать время отправки.

Выйти из первого ящика.

Войти как:

user2@company.test

Проверить:

- письмо появилось во входящих;
- отправитель определён правильно;
- тема совпадает;
- содержимое читается;
- время доставки не имеет значительной задержки.

Ответить пользователю:

user1@company.test

с темой:

RE: LAB11 internal mail flow

Войти в первый ящик и проверить получение ответа.

Полный внутренний Mail Flow

```
user1 MUA
  ↓ Submission
Postfix
  ↓ Filtering
Rspamd
  ↓ Local Delivery
Dovecot / Mail Storage
  ↓ Webmail или IMAPS
user2 MUA
```

Работоспособность подтверждается получением письма, а не только состоянием контейнеров.

Задание 7. Проверить SMTP, Submission, IMAPS и HTTPS

SMTP 25 с STARTTLS

На client01:

```
openssl s_client \  
-starttls smtp \  
-connect mail.company.test:25 \  
-servername mail.company.test \  
-brief
```

Для выхода после установления соединения:

Ctrl+C

Проверить:

- TLS установился;
 - сервер показал SMTP Banner;
 - сертификат относится к mail.company.test;
 - соединение не завершается протокольной ошибкой.
-

Submission 587

```
openssl s_client \  
-starttls smtp \  
-connect mail.company.test:587 \  
-servername mail.company.test \  
-brief
```

Submission должен поддерживать:

- STARTTLS;
 - SMTP Authentication;
 - отправку от имени авторизованного пользователя.
-

IMAPS 993

```
openssl s_client \  
-connect mail.company.test:993 \  
-servername mail.company.test \  
-brief
```

Проверить:

- успешное TLS-соединение;
- сертификат;
- IMAP Banner.

HTTPS 443

```
curl -I https://mail.company.test/
```

Если в стенде используется недоверенный клиенту учебный сертификат, для диагностического теста допускается:

```
curl -kI https://mail.company.test/
```

Ключ -k используется только в изолированной лаборатории и не является исправлением ошибки доверия TLS.

Проверка сертификата

```
openssl s_client \  
  -connect mail.company.test:993 \  
  -servername mail.company.test \  
  </dev/null 2>/dev/null |  
openssl x509 \  
  -noout \  
  -subject \  
  -issuer \  
  -dates \  
  -ext subjectAltName
```

В Subject Alternative Name должно присутствовать:

DNS:mail.company.test

Задание 8. Проверить SMTP Authentication и IMAPS Login

Отправка через Submission

На client01 выполнить:

```
swaks \  
  --server mail.company.test \  
  --port 587 \  
  --tls \  
  --auth \  
  --auth-user user1@company.test \  
  --from user1@company.test \  
  --to user2@company.test \  
  --header 'Subject: LAB11 swaks submission test' \  
  --body 'Message sent through authenticated Submission'
```

Пароль вводится интерактивно по запросу программы.

Пароль нельзя:

- указывать прямо в команде;
- добавлять в отчёт;
- сохранять в Shell History;
- показывать на скриншоте.

В выводе найти SMTP-коды:

```
235
250
```

Код 235 обычно подтверждает успешную Authentication, а 250 — принятие соответствующей SMTP-команды.

Войти в `user2@company.test` и убедиться, что письмо доставлено.

Проверка IMAPS Login

Выполнить:

```
curl \
  --url 'imaps://mail.company.test/INBOX/' \
  --user 'user2@company.test' \
  --ssl-reqd
```

`curl` запросит пароль интерактивно.

Если используется недоверенный учебный сертификат:

```
curl \
  --insecure \
  --url 'imaps://mail.company.test/INBOX/' \
  --user 'user2@company.test' \
  --ssl-reqd
```

Ожидаемый результат

Команда получает список или метаданные Mailbox без ошибки Authentication.

Задание 9. Проверить очередь и журналы

Перейти в каталог Mailcow:

```
cd /opt/mailcow-dockerized
```

Проверить очередь Postfix:

```
sudo docker compose exec \
  "$POSTFIX_SERVICE" \
  postqueue -p
```

При нормальном внутреннем Mail Flow очередь обычно пустая:

Mail queue is empty

Если в очереди есть сообщения, записать:

- Queue ID;
- отправителя;
- получателя;
- причину задержки;
- время нахождения в очереди.

Посмотреть журналы Postfix:

```
sudo docker compose logs \
  --since 15m \
  "$POSTFIX_SERVICE"
```

Посмотреть журналы Dovecot:

```
sudo docker compose logs \
  --since 15m \
  "$DOVECOT_SERVICE"
```

Посмотреть журналы Rspamd:

```
sudo docker compose logs \
  --since 15m \
  "$RSPAMD_SERVICE"
```

Показать только последние строки:

```
sudo docker compose logs \
  --tail 50 \
  "$POSTFIX_SERVICE" \
  "$DOVECOT_SERVICE" \
  "$RSPAMD_SERVICE"
```

Безопасная работа с очередью

В базовой работе разрешено:

```
postqueue -p
```

Для конкретного Queue ID преподаватель может разрешить:

```
postcat -q QUEUE_ID
```

Нельзя без анализа причины выполнять:

```
postqueue -f
postsuper -d ALL
```

Flush или массовое удаление не исправляют причину появления Deferred Messages.

Задание 10. Проверить запрет Open Relay

Open Relay означает, что внешний неаутентифицированный отправитель может пересылать почту через сервер на чужой внешний домен.

В лаборатории используется безопасная проверка только до команды RCPT TO.

Письмо фактически не отправляется.

Проверка собственного домена

На client01:

```
swaks \
  --server mail.company.test \
  --port 25 \
  --from outsider@example.test \
  --to user2@company.test \
  --quit-after RCPT
```

Сервер может принять получателя, потому что:

company.test

является обслуживаемым локальным доменом.

Проверка Relay на внешний домен

```
swaks \
  --server mail.company.test \
  --port 25 \
  --from outsider@example.test \
  --to recipient@external.test \
  --quit-after RCPT
```

Ожидается отказ вида:

550 Relay access denied

или другое постоянное SMTP-отклонение неавторизованного Relay.

Матрица доверия

Отправитель	Получатель	Ожидаемый результат
Внешний неавторизованный	user2@company.test	Приём для собственного домена возможен
Авторизованный	Внешний домен	Relay разрешается по

Отправитель	Получатель	Ожидаемый результат
пользователь		политике
Внешний неавторизованный	Внешний домен	Relay запрещён

Приём письма для собственного домена не является Open Relay.

Задание 11. Исследовать заголовки и антиспам

Открыть в Webmail исходный код письма:

```
LAB11 swaks submission test
```

Найти заголовки:

```
Received
Authentication-Results
Received-SPF
DKIM-Signature
X-Spam-*
```

Точный набор заголовков зависит от:

- версии Mailcow;
- конфигурации Rspamd;
- внутренних DNS-записей;
- способа отправки;
- политики домена.

В отчёте указать:

Заголовок	Обнаружен	Назначение
Received	Да/нет	Путь письма
Authentication-Results	Да/нет	SPF, DKIM и DMARC
Received-SPF	Да/нет	Результат SPF
DKIM-Signature	Да/нет	Доменная подпись
X-Spam-*	Да/нет	Антиспам-оценка

Важно

SPF, DKIM и DMARC:

- не шифруют содержимое письма;
- не гарантируют попадание во входящие;
- не заменяют TLS;
- не заменяют антиспам;
- не отменяют необходимость анализа журналов и очереди.

Задание 12. Смоделировать отказ Dovecot

Перед отказом проверить исходное состояние:

```
sudo docker compose ps
```

На client01 проверить IMAPS:

```
openssl s_client \  
-connect mail.company.test:993 \  
-servername mail.company.test \  
-brief
```

Проверить вход в Webmail и открытие Mailbox.

Остановить Dovecot

На сервере Mailcow:

```
sudo docker compose stop \  
"$DOVECOT_SERVICE"
```

Проверить:

```
sudo docker compose ps
```

Проверить порт:

```
sudo ss -lntp |  
grep ':993'
```

Повторить на client01:

```
openssl s_client \  
-connect mail.company.test:993 \  
-servername mail.company.test \  
-brief
```

Проверить Webmail:

- сама Web-страница может открываться;
- вход или чтение Mailbox должны нарушиться;
- SMTP-контур может вести себя иначе, чем IMAP;
- точное поведение локальной доставки зависит от версии и архитектуры платформы.

Посмотреть журналы:

```
sudo docker compose logs \  
--since 10m \  
"$DOVECOT_SERVICE"
```

Посмотреть связанные ошибки:

```
sudo docker compose logs \
  --since 10m |
  grep -i -E \
    'dovecot|imap|lmtp|authentication|error|failed|deferred'
```

Проверить очередь:

```
sudo docker compose exec \
  "$POSTFIX_SERVICE" \
  postqueue -p
```

Зафиксировать

- какой пользовательский симптом возник;
 - доступен ли HTTPS;
 - доступен ли SMTP;
 - доступен ли IMAPS;
 - появились ли Deferred Messages;
 - какие строки журнала подтверждают причину.
-

Восстановить Dovecot

Запустить контейнер:

```
sudo docker compose start \
  "$DOVECOT_SERVICE"
```

Проверить:

```
sudo docker compose ps
```

Подождать готовность сервиса:

```
sleep 10
```

Проверить IMAPS:

```
openssl s_client \
  -connect mail.company.test:993 \
  -servername mail.company.test \
  -brief
```

Войти в Webmail.

Отправить новое письмо:

user1@company.test → user2@company.test

Тема:

LAB11 mail flow after Dovecot recovery

Проверить:

- отправку;
- локальную доставку;
- появление письма во входящих;
- чтение письма;
- состояние очереди.

Исправление считается подтверждённым только после полного повторного Mail Flow Test.

8. Приёмочная матрица

№	Проверка	Ожидаемый результат	Фактический результат
1	hostname -f	mail.company.test	
2	A-запись	192.168.30.20	
3	MX-запись	10 mail.company.test	
4	Mailcow Containers	Основные роли Running	
5	Webmail Login user1	Успешно	
6	Webmail Login user2	Успешно	
7	user1 → user2	Письмо доставлено	
8	user2 → user1	Ответ доставлен	
9	SMTP 25	Banner и STARTTLS	
10	Submission 587	TLS и Authentication	
11	IMAPS 993	TLS и Login	
12	HTTPS	Webmail доступен	
13	Очередь	Нет необъяснимых Deferred Messages	
14	Open Relay	Неавторизованный Relay запрещён	
15	Dovecot Stop	IMAPS и Mailbox Access нарушены	
16	Dovecot Recovery	IMAPS восстановлен	
17	Итоговый Mail Flow	Письмо доставляется и читается	

9. Дополнительные задания

Дополнительное задание 1. Проверить SPF, DKIM и DMARC

На client01 проверить SPF:

```
dig @192.168.30.53 \
    company.test \
    TXT
```

Учебная запись может иметь вид:

```
v=spf1 mx -all
```

Проверить DMARC:

```
dig @192.168.30.53 \
    _dmarc.company.test \
    TXT
```

Начальная политика:

```
v=DMARC1; p=none
```

В административном интерфейсе Mailcow определить DKIM Selector и получить Public Key.

Проверить запись:

```
dig @192.168.30.53 \
    <selector>._domainkey.company.test \
    TXT
```

Private DKIM Key:

- остаётся внутри почтовой платформы;
- не копируется в отчёт;
- не сохраняется в обычном Git-репозитории;
- включается в защищённый Backup.

Отправить новое письмо и проверить:

```
DKIM-Signature
Authentication-Results
```

Вывод

DMARC требует Alignment домена SPF или DKIM с доменом видимого поля From.

Дополнительное задание 2. Подготовить план Backup и Test Restore

Зафиксировать:

```
git describe --tags --always
sudo docker version
sudo docker compose version
sudo docker compose ps
```

Для Backup использовать только штатную процедуру, совместимую с установленной версией Mailcow.

Команду или сценарий предоставляет преподаватель после проверки документации выбранной версии.

Backup должен включать:

- Mailboxes;
- Database;
- конфигурацию Mailcow;
- Compose-файлы;
- Environment Configuration;
- TLS Certificates и Private Keys;
- DKIM Private Keys;
- необходимые Secrets;
- пользовательские настройки;
- антиспам-политику;
- сведения о версии.

Backup должен:

- храниться в зашифрованном виде;
- иметь ограниченные права;
- храниться отдельно от рабочей VM;
- иметь Retention Policy;
- проверяться восстановлением.

Test Restore выполняется:

- на отдельной чистой VM;
- с совместимой версией Docker;
- с совместимой версией Mailcow;
- без публикации восстановленных данных во внешнюю сеть.

После восстановления проверить:

1. запуск контейнеров;
2. вход в Web UI;
3. наличие домена;
4. наличие Mailbox;
5. письма в Mailbox;
6. отправку;
7. получение;
8. очередь;

- 9. DNS;
- 10. TLS.

Дополнительное задание 3. Спроектировать мониторинг Mailcow

Составить матрицу мониторинга.

Уровень	Метрика или проверка
Host	CPU, RAM, Load, Disk, Inode, I/O
Docker	Docker Daemon, Running/Exited, Restart Count
SMTP	TCP 25 и SMTP Banner
Submission	TCP 587, STARTTLS, Authentication
IMAPS	TCP 993, TLS и Login
HTTPS	Webmail Response
Queue	Размер очереди и возраст старейшего сообщения
Mail Flow	Отправка и получение тестового письма
TLS	Дни до истечения сертификата
Backup	Возраст последнего успешного Backup
Storage	Mail Storage, Database Volume, Queue Storage
Dependencies	DNS, NTP, Database и Redis

Для Zabbix разделить Triggers:

- один контейнер остановлен;
- Postfix недоступен;
- Dovecot недоступен;
- очередь растёт;
- есть Deferred Messages;
- сертификат истекает менее чем через 14 дней;
- Backup старше RPO;
- полный Synthetic Mail Flow Test завершился ошибкой.

10. Основные команды диагностики

Сеть и DNS

```
hostname -f
ip -br addr
ip route
getent hosts mail.company.test
```

```
dig mail.company.test A
dig company.test MX
```

Контейнеры

```
sudo docker compose ps
sudo docker compose config --services
sudo docker compose logs --tail 100
```

Порты

```
sudo ss -lntup |
  grep -E ':(25|443|465|587|993)\b'
```

TLS

```
openssl s_client \
  -starttls smtp \
  -connect mail.company.test:587 \
  -servername mail.company.test
```

```
openssl s_client \
  -connect mail.company.test:993 \
  -servername mail.company.test
```

SMTP

```
swaks \
  --server mail.company.test \
  --port 587 \
  --tls \
  --auth \
  --auth-user user1@company.test \
  --to user2@company.test
```

Очередь

```
sudo docker compose exec \
  "$POSTFIX_SERVICE" \
  postqueue -p
```

11. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему и таблицу адресации.
3. FQDN сервера.
4. Версии Mailcow, Docker и Compose.
5. Список основных контейнерных ролей.

6. Результаты проверки A и MX.
7. Созданный Mail Domain.
8. Созданные адреса Mailbox.
9. Результат внутреннего Mail Flow.
10. Результаты проверки SMTP, Submission и IMAPS.
11. Результат проверки очереди.
12. Результат Open Relay Test.
13. Найденные почтовые заголовки.
14. Симптомы отказа Dovecot.
15. Результат восстановления Dovecot.
16. Заполненную приёмочную матрицу.
17. Краткий вывод по работе.

В отчёт запрещено включать:

- пароли пользователей;
 - пароль администратора;
 - TLS Private Keys;
 - DKIM Private Keys;
 - Administrative Tokens;
 - полное содержимое файлов с Secrets;
 - содержимое почтовых сообщений, содержащее личные данные;
 - полный Backup Mailcow.
-

12. Чек-лист выполнения

- ☐ `hostname -f` возвращает `mail.company.test`.
- ☐ Системное время синхронизировано.
- ☐ A-запись возвращает `192.168.30.20`.
- ☐ MX указывает на `mail.company.test`.
- ☐ Основные контейнеры Mailcow запущены.
- ☐ Зафиксированы версии платформы.
- ☐ Создан домен `company.test`.
- ☐ Создан `user1@company.test`.
- ☐ Создан `user2@company.test`.
- ☐ Пользователь `user1` входит в Webmail.
- ☐ Пользователь `user2` входит в Webmail.
- ☐ Письмо `user1 → user2` доставляется.
- ☐ Ответ `user2 → user1` доставляется.
- ☐ SMTP 25 отвечает.

- ☐ Submission 587 поддерживает TLS и Authentication.
 - ☐ IMAPS 993 поддерживает TLS и Login.
 - ☐ TLS-сертификат содержит mail.company.test.
 - ☐ Очередь не содержит необъяснимых сообщений.
 - ☐ Open Relay запрещён.
 - ☐ Исследованы заголовки письма.
 - ☐ Смоделирован отказ Dovecot.
 - ☐ Ошибка подтверждена журналами и протокольным тестом.
 - ☐ Dovecot восстановлен.
 - ☐ Выполнен итоговый Mail Flow Test.
 - ☐ Пароли и ключи не включены в отчёт.
-

13. Контрольные вопросы

1. Чем SMTP 25 отличается от Submission 587 и 465?
2. Какие роли выполняют MUA, MTA и MDA?
3. Почему Dovecot нельзя называть только хранилищем писем?
4. Почему MX должен указывать на имя сервера, а не непосредственно на IP?
5. Чем SPF, DKIM и DMARC отличаются друг от друга?
6. Почему приём письма для собственного домена не является Open Relay?
7. Чем успешная проверка порта отличается от полного Mail Flow Test?
8. Почему активные Volumes Mailcow нельзя универсально копировать как обычные файлы?